

Chapter 5: Interfaces, Implementations, and Polymorphism

Fundamentals of Python: Data Structures, Second Edition

Learning Objectives

- Develop an interface for a given collection type
- Implement multiple classes that conform to the interface of a collection type
- Assess the tradeoffs in running time and memory usage of multiple implementations of a given collection type
- Implement a basic iterator
- Use methods to manipulate bags and sets
- Decide whether a bag or a set is an appropriate collection for a given application
- Convert a bag implementation to a sorted bag implementation

Developing an Interface

- When you run Python's help function to obtain information about a module, data type, method, or function,
 - You are accessing documentation about that resource's interface
- Interfaces are concise and informative:
 - They allow you to grasp the behavior of a resource
- In this section, you develop the interface for a simple collection type called a bag:
 - A bag interface allows clients to use bags effectively and allows implementers to produce new classes that implement this interface

Designing the Bag Interface (1 of 2)

- Derive an interface from thinking about what bags can do in a real-world situation:
 - A bag can contain any objects
- You will want to know the following:
 - How many things are in a bag
 - How to add things to it or remove them from it
 - Whether a bag is empty
 - How to empty a bag in a single operation
 - How to determine whether a given item is in a bag
 - How to view each item in a bag without emptying it
 - How to create a bag

Designing the Bag Interface (2 of 2)

- The next step is to draw up a list of function names, method names, and operator symbols that meet the descriptions of these operations
- Examples include:
 - `add`
 - `clear`
 - `count`
 - `for ...`
 - `in`
 - `isEmpty`
 - `len`
 - `remove`
 - `str`
 - `+`
 - `==`

Specifying Arguments and Return Values (1 of 3)

- Next, add arguments to the operations in the interface
 - And think about what values, if any, they return
- Code where the variables **b** and **c** refer to bags:

```
b.clear()                # Make the bag empty
for item in range(10):   # Add 10 numbers to it
    b.add(item)
print(b)                 # Print the contents (a string)
print(4 in b)            # Is 4 in the bag?
print(b.count(2))        # 1 instance of 2 in the bag
c = b + b                # Contents replicated in a new bag
print(len(c))            # 20 numbers
for item in c:           # Print them all individually
    print(item)
for item in range(10):   # Remove half of them
    c.remove(item)
print(c == b)            # Should be the same
contents now
```

Specifying Arguments and Return Values (2 of 3)

- Table 5.1 Arguments for Bag Operations and Their Methods

User's Bag Operation	Method in a Bag Class
<code>b = <class name> (<optional collection>)</code>	<code>__init__(self, sourceCollection = None)</code>
<code>b.add(item)</code>	<code>add(self, item)</code>
<code>b.clear()</code>	<code>clear(self)</code>
<code>b.count(item)</code>	<code>count(self, item)</code>
<code>b.isEmpty()</code>	<code>isEmpty(self)</code>
<code>b.remove(item)</code>	<code>remove(self, item)</code>
<code>len(b)</code>	<code>__len__(self)</code>
<code>str(b)</code>	<code>__str__(self)</code>
<code>for item in b:</code>	<code>__iter__(self)</code>

Specifying Arguments and Return Values (3 of 3)

- Table 5.1 Arguments for Bag Operations and Their Methods (continued)

User's Bag Operation	Method in a Bag Class
<code>item in b</code>	<code>__contains__(self, item)</code>
	Not needed if <code>__iter__</code> is included
<code>b1 + b2</code>	<code>__add__(self, other)</code>
<code>b == anyObject</code>	<code>__eq__(self, other)</code>

Constructors and Implementing Classes

- The first row in Table 5.1 shows an operation that is the constructor for the particular type of bag being created
- The syntactic form **<class name>** is used in Table 5.1 to indicate that this can be the name of any implementing bag class:
 - The method in the right column is always named **`_init_`**
- The code segment that creates an empty linked bag and an array bag that contains the numbers in a given list is as follows:

```
from arraybag import ArrayBag
from linkedbag import LinkedBag
```

```
bag1 = LinkedBag()
bag2 = ArrayBag([20, 60, 100, 43])
```

Preconditions, Postconditions, Exceptions, and Documentation (1 of 2)

- Final step before expressing an interface in code:
 - To describe clearly and concisely what each method does
 - Includes not only what you expect to occur under normal conditions but also what will happen when something abnormal, such as an error, occurs
- A more detailed form of documentation can include preconditions and postconditions:
 - Precondition is a statement of what must be true for a method to perform its actions correctly
 - Postcondition states what will be true after the method completes execution, assuming that its preconditions are also true:
 - Usually included in mutator methods, which modify the internal state of an object

Preconditions, Postconditions, Exceptions, and Documentation (2 of 2)

- Documentation in an interface should include a statement of any exceptions that could be raised:
 - Example, a bag's **remove** method might raise a **KeyError** if the target item is not in the bag
- A Python method header for the remove method:

```
def remove(self, item):  
    """Precondition: item is in self.  
    Raises: KeyError if item is not in self.  
    Postcondition: item is removed from self."""
```

Coding an Interface in Python (1 of 3)

- To create an interface, list each of the method headers with its documentation and complete each method with a single **pass** or **return** statement:
 - A **pass** statement is used in the mutator methods that return no value, whereas each accessor method returns a simple default value, such as **False**, **0**, or **None**
- So method headers can be checked with the compiler
 - Place them within a class whose suffix is “Interface”

Coding an Interface in Python (2 of 3)

- Listing of the bag interface, as defined in the class **BagInterface**:

```
"""
File: baginterface.py
Author: Ken Lambert
"""

class BagInterface(object):
    """Interface for all bag types."""

    # Constructor
    def __init__(self, sourceCollection=None):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        pass

    # Accessor methods
    def isEmpty(self):
        """Returns True if len(self) == 0,
        or False otherwise."""
        return True

    def __len__(self):
        """Returns the number of items in self."""
        return 0

    def __str__(self):
        """Returns the string representation of self."""
        return ""

    def __iter__(self):
        """Supports iteration over a view of self."""
        return None
```

Coding an Interface in Python (3 of 3)

- Listing of the bag interface, as defined in the class **BagInterface** (continued):

```
def __add__(self, other):  
    """Returns a new bag containing the contents  
    of self and other."""  
    return None  
  
def __eq__(self, other):  
    """Returns True if self equals other,  
    or False otherwise."""  
    return False  
  
def count(self, item):  
    """Returns the number of instances of item in self."""  
    return 0  
  
# Mutator methods  
def clear(self):  
    """Makes self become empty."""  
    pass  
  
def add(self, item):  
    """Adds item to self."""  
    pass  
  
def remove(self, item):  
    """Precondition: item is in self.  
    Raises: KeyError if item is not in self.  
    Postcondition: item is removed from self."""  
    Pass
```

Developing an Array-Based Implementation

- This section covers developing an array-based implementation of the bag interface, called **ArrayBag**
- The design and implementation of the class consists of two steps:
 - Choose an appropriate data structure to contain the collection's items and determine any other data that might be needed to represent the state of the collection (these data are assigned to instance variables in the `_init_` method)
 - Complete the code for the methods specified in the interface

Choose and Initialize the Data Structures (1 of 3)

- An array-based implementation
 - Each object of type **ArrayBag** contains an array of the items in the bag
 - Can be an instance of the **Array** class or another array-based collection, such as Python's **list** type
- After initializing the two instance variables, the `__init__` method must deal with the possibility that its caller has provided a source collection parameter:
 - If that is the case, all the data in the source collection must be added to the new **ArrayBag** object

Choose and Initialize the Data Structures (2 of 3)

- The code for this part of the design is easy to create
- You just make a copy of the bag interface file, **baginterface.py**, and rename it to **arraybag.py**
- You then add an **import** statement for the array, rename the class to **ArrayBag**, add a class variable for the default capacity, and complete the **__init__** method

Choose and Initialize the Data Structures (3 of 3)

```
"""
File: arraybag.py
Author: Ken Lambert
"""

from arrays import Array

class ArrayBag(object):
    """An array-based bag implementation."""

    # Class variable
    DEFAULT_CAPACITY = 10

    # Constructor
    def __init__(self, sourceCollection = None):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        self.items = Array(ArrayBag.DEFAULT_CAPACITY)
        self.size = 0
        if sourceCollection:
            for item in sourceCollection:
                self.add(item)
```

Complete the Easy Methods First (1 of 2)

- Simplest methods in this interface:
 - `IsEmpty`, `__len__`, `clear`, and `add`
- Code for these four methods:

```
# Accessor methods
def isEmpty(self):
    """Returns True if len(self) == 0, or False otherwise."""
    return len(self) == 0

def __len__(self):
    """Returns the number of items in self."""
    return self.size

# Mutator methods
def clear(self):
    """Makes self become empty."""
    self.size = 0
    self.items = Array(ArrayBag.DEFAULT_CAPACITY)

def add(self, item):
    """Adds item to self."""
    # Check array memory here and increase it if necessary
    self.items[len(self)] = item
    self.size += 1
```

Complete the Easy Methods First (2 of 2)

- You should call a method or function on **self** to get something done within a class definition, whenever possible
- For example,
 - Whenever you need to use the logical size of the bag within a class definition, run **len(self)** instead of referring directly to the instance variable **self.size**
- The increment of **size** in the **add** method
 - Requires you to make the variable the target of the assignment; so you cannot avoid referring to it at that point
- The **add** method places the new item at the logical end of the array

Complete the Iterator (1 of 2)

- The new `__iter__` method maintains a cursor that allows it to navigate through a sequence of objects:
 - The caller's `for` loop drives this process
- On each pass through the calling `for` loop,
 - The item at the cursor is yielded to the caller, and then the cursor is advanced to the next object in the sequence
 - When the cursor reaches the length of the bag, the `__iter__` method's `while` loop terminates
 - Which in turn terminates the calling `for` loop

Complete the Iterator (2 of 2)

- Code for the `__iter__` method:

```
def __iter__(self):  
    """Supports iteration over a view of self."""  
    cursor = 0  
    while cursor < len(self):  
        yield self.items[cursor]  
        cursor += 1
```

Complete the Methods That Use the Iterator (1 of 2)

- The `__eq__` method follows the rules for the equality test discussed in Chapter 2
- The `__add__` method follows the rules for the concatenation of two collections discussed in Chapter 2
- The `__str__` method uses the `map` and `join` operations to build a string containing the string representations of the bag's items

Complete the Methods That Use the Iterator (2 of 2)

- Code for these methods:

```
def __str__(self):  
    """Returns the string representation of self."""  
    return "{" + ", ".join(map(str, self)) + "}"  
  
def __add__(self, other):  
    """Returns a new bag containing the contents  
    of self and other."""  
    result = ArrayBag(self)  
    for item in other:  
        result.add(item)  
    return result  
  
def __eq__(self, other):  
    """Returns True if self equals other,  
    or False otherwise."""  
    if self is other: return True  
    if type(self) != type(other) or \  
        len(self) != len(other):  
        return False  
    for item in self:  
        if self.count(item) != other.count(item):  
            return False  
    return True
```

The `in` Operator and the `__contains__` Method

- When Python sees the `in` operator used with a collection,
 - It runs the `__contains__` method in the collection's class
- However, if the author of this class does not include such a method,
 - Python automatically generates a default method
 - This method performs a simple sequential search for the target item, using a `for` loop on `self`
- Because the search of a bag can be no better than linear on the average,
 - You rely on the default implementation of `__contains__`
 - Omitting the method for it in the `ArrayBag` class

Complete the `remove` Method (1 of 2)

- The `remove` method is the most challenging method to complete in the bag implementation
- To begin with, you must check the precondition and raise an exception if it is violated
- Then you must search the underlying array for the target item
- Finally, you must shift the items in the array to the left to close the hole left by the removed item:
 - Decrement the bag's size by one
 - Resize the array if necessary

Complete the remove Method (2 of 2)

- Code for the **remove** method:

```
def remove(self, item):  
    """Precondition: item is in self.  
    Raises: KeyError if item is not in self.  
    postcondition: item is removed from self."""  
    # 1. check precondition and raise an exception if necessary  
    if not item in self:  
        raise KeyError(str(item) + " not in bag")  
    # 2. Search for index of target item  
    targetIndex = 0  
    for targetItem in self:  
        if targetItem == item:  
            break  
        targetIndex += 1  
    # 3. Shift items to the right of target left by one position  
    for i in range(targetIndex, len(self) - 1):  
        self.items[i] = self.items[i + 1]  
    # 4. Decrement logical size  
    self.size -= 1  
    # 5. Check array memory here and decrease it if necessary
```

Developing a Link-Based Implementation (1 of 2)

- To develop a link-based implementation of a bag collection, focus on two things:
 - You're using the same interface as before, with all the methods specified in the file **baginterface.py**
 - You need to shift your way of thinking from arrays to linked structures to represent the bag's data
- If a method does not access the array variable, it does not have to access the linked structure variable either
- Therefore, you can copy several completed methods from the **ArrayBag** class, without changes to the **LinkedBag** class

Developing a Link-Based Implementation (2 of 2)

- The only methods that will have different implementations in the **LinkedList** class are those that cannot avoid this direct access to data:
 - `__init__`
 - `__iter__`
 - `clear`
 - `add`
 - `remove`

Initialize the Data Structures (1 of 2)

- The role of the `__init__` method in **LinkedList** is to create the instance variables and give them initial values
- In this case, instead of an array and a logical size, the two pieces of data are a linked structure and a logical size
- To maintain consistency, you can use the same variable names as before:
 - However, **`self.items`** is now an external pointer instead of an array
 - This pointer is initially set to **`None`**, the state of an empty linked structure
 - When the structure is not empty, **`self.items`** refers to the first node in the linked structure

Initialize the Data Structures (2 of 2)

- Code:

```
"""
File: linkedbag.py
Author: Ken Lambert
"""

from node import Node

class LinkedBag(object):
    """A link-based bag implementation."""

    # Constructor
    def __init__(self, sourceCollection = None):
        """Sets the initial state of self, which includes the
        contents of sourceCollection, if it's present."""
        self.items = None
        self.size = 0
        if sourceCollection:
            for item in sourceCollection:
                self.add(item)
```

Complete the Iterator

- The `__iter__` method for **LinkedBag** supports the same kind of traversal as it does in **ArrayBag**:
 - The logical structure of the two methods is quite similar
- Code:

```
def __iter__(self):  
    """Supports iteration over a view of self."""  
    cursor = self.items  
    while cursor != None:  
        yield cursor.data  
        cursor = cursor.next
```

Complete the Methods `clear` and `add`

- The method `clear` in `LinkedBag` is quite similar to its sister method in `ArrayBag`
- The method `add` in `LinkedBag` leverages constant-time access by placing the new item at the head of the linked structure
- Code:

```
def add(self, item):  
    """Adds item to self."""  
    self.items = Node(item, self.items)  
    self.size += 1
```

Complete the Method `remove` (1 of 2)

- The method `remove` in `LinkBag` must first handle the precondition and then do a sequential search for the target item
- When the node containing the target item is found, there are two cases to consider:
 - The target node is at the head of the linked structure, in which case, you must reset variable `self.items` to this node's next link
 - The target node is some node after the first one; in which case, the next link of the node before it must be reset to the next link of the target node

Complete the Method `remove` (2 of 2)

```
def remove(self, item):
    """Precondition: item is in self.
    Raises: KeyError if item is not in self.
    Postcondition: item is removed from self."""
    # Check precondition and raise an exception if necessary
    if not item in self:
        raise KeyError(str(item) + " not in bag")
    # Search for the node containing the target item
    # probe will point to the target node, and trailer
    # will point to the node before it, if it exists
    probe = self.items
    trailer = None
    for targetItem in self:
        if targetItem == item:
            break
        trailer = probe
        probe = probe.next
    # Unhook the node to be deleted, either the first one or
    # one thereafter
    if probe == self.items:
        self.items = self.items.next
    else:
        trailer.next = probe.next
    # Decrement logical size
    self.size -= 1
```

Run-Time Performance of the Two Bag Implementations (1 of 2)

- The running times of the operations on the two implementations of bags are quite similar:
 - The **in** and **remove** operations take linear time in both implementations, because they incorporate a sequential search
 - The **remove** operation in **ArrayBag** must do the additional work of shifting data items in the array, but the cumulative effect is not worse than linear
 - The **+**, **str**, and **iter** operations are linear
- The running time of the **==** operation has several cases and is left as an exercise
- The remaining operations are constant time:
 - Although **ArrayBag**'s **add** incurs an occasional linear time hit to resize the array

Run-Time Performance of the Two Bag Implementations (2 of 2)

- The two implementations have the expected memory tradeoffs
- When the array within an **ArrayBag** is better than half full,
 - It uses less memory than a **LinkedBag** of the same logical size
- In the worst case,
 - A **LinkedBag** uses twice as much memory as an **ArrayBag** whose array is full
- Because of these memory tradeoffs, removals from an **ArrayBag** are generally slower than they are on a **LinkedBag**

Testing the Two Bag Implementations (1 of 3)

- The approach used to test these implementations is to include a tester function for each resource that you develop
- This function both
 - Ensures that a new collection class conforms to a collection interface
 - Checks to see that these operations do what they are supposed to do

Testing the Two Bag Implementations (2 of 3)

- Code for a standalone application that you can run with any bag class
 - The `main` function expects a type as an argument and runs the tests on objects of that type

```
"""
File: testbag.py
Author: Ken Lambert
A tester program for bag implementations.
"""

from arraybag import ArrayBag
from linkedbag import LinkedBag

def test(bagType):
    """Expects a bag type as an argument and runs some tests
    on objects of that type."""
    print("Testing", bagType)
    lyst = [2013, 61, 1973]
    print("The list of items added is:", lyst)
    b1 = bagType(lyst)
    print("Length, expect 3:", len(b1))
    print("Expect the bag's string:", b1)
    print("2013 in bag, expect True:", 2013 in b1)
    print("2012 in bag, expect False:", 2012 in b1)
    print("Expect the items on separate lines:")
    for item in b1:
        print(item)
```

Testing the Two Bag Implementations (3 of 3)

- Code (continued):

```
b1.clear()
print("Clearing the bag, expect {}:".format(b1))
b1.add(25)
b1.remove(25)
print("Adding and then removing 25, expect {}:".format(b1))
b1 = bagType(lyst)
b2 = bagType(b1)
print("Cloning the bag, expect True for ==:".format(b1 == b2))
print("Expect False for is:".format(b1 is b2))
print("+ the two bags, expect two of each item:".format(b1 + b2))
for item in lyst:
    b1.remove(item)
print("Remove all items, expect {}:".format(b1))
print("Removing nonexistent item, expect crash with KeyError:")
b2.remove(99)
```

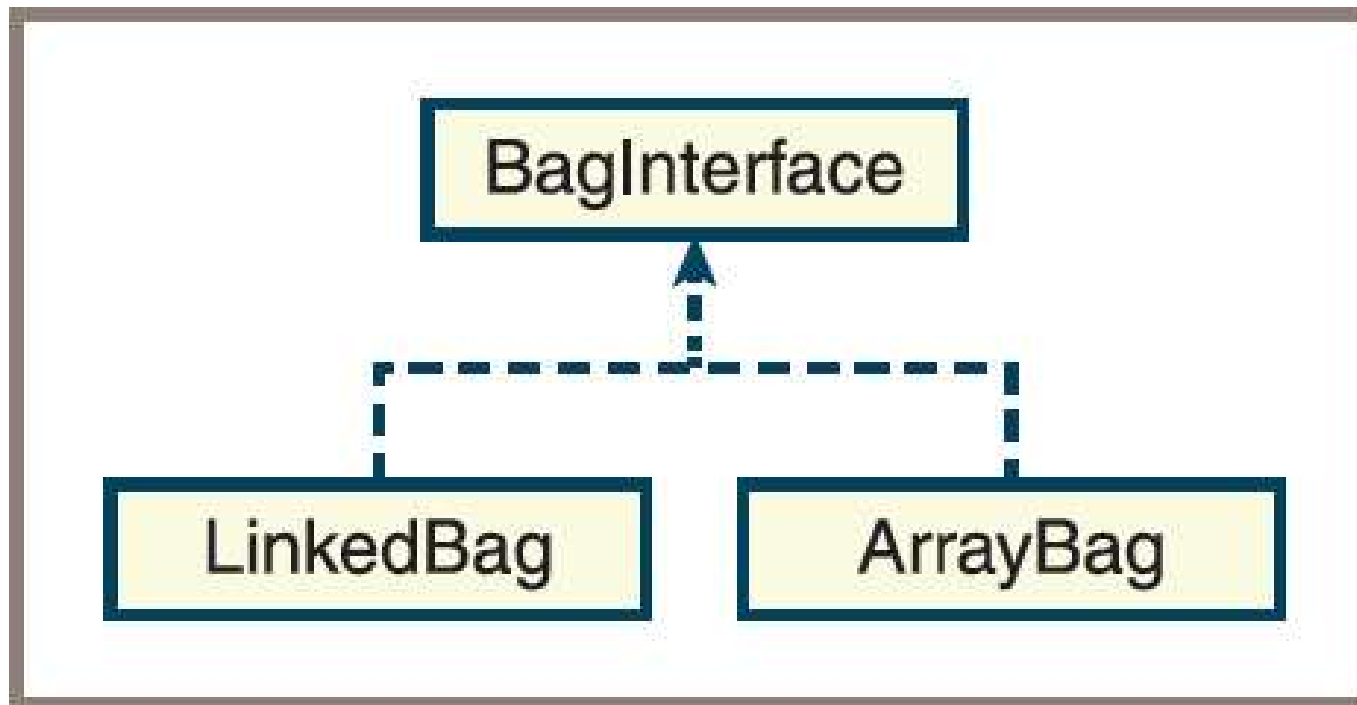
```
test(ArrayBag)
# test(LinkedBag)
```

Diagramming the Bag Resource with UML (1 of 3)

- It will help to catalog resources with a set of visual aids called *class diagrams*:
 - These diagrams are taken from a visual language called Unified Modeling Language (UML)
- Class diagrams show the relationships among classes at various levels of detail

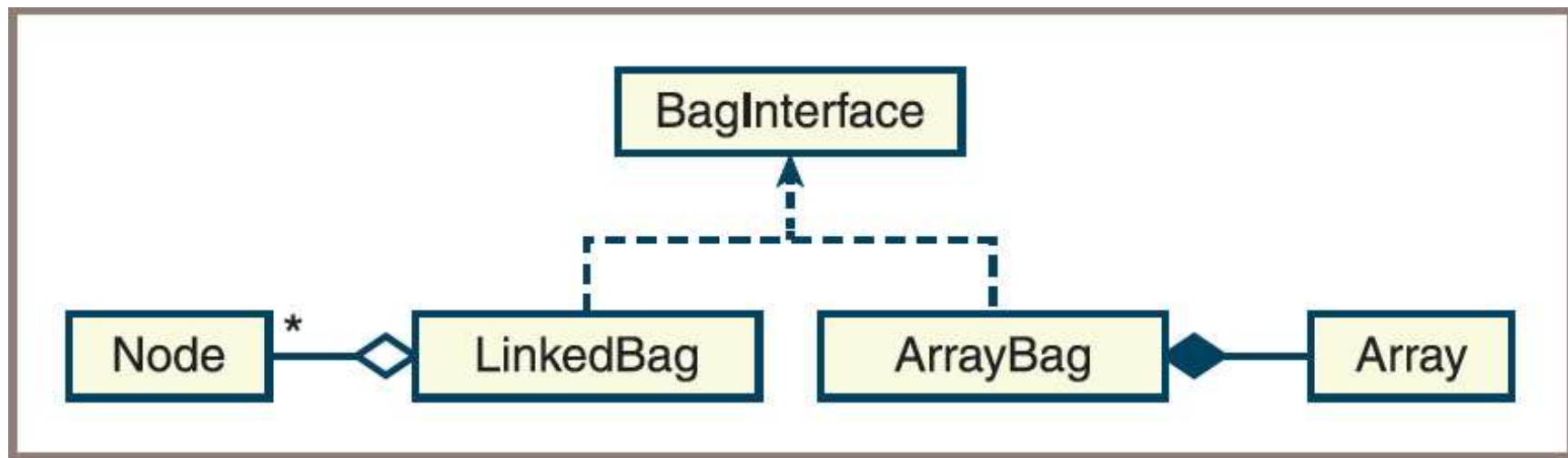
Diagramming the Bag Resource with UML (2 of 3)

Figure 5-1: A class diagram with an interface and two implementing classes



Diagramming the Bag Resource with UML (3 of 3)

Figure 5-2: A class diagram with aggregation and composition relationships



Chapter Summary (1 of 3)

- An interface is the set of operations available to the user of a software resource
- The items in an interface are the headers of functions and methods, together with their documentation
- Preconditions state what must be true before a function or method can complete its task correctly
- Postconditions state what must be true after a function or method completes its task completely
- A well-designed software system separates the interfaces from their implementations
- An implementation is a function, method, or class that conforms to an interface

Chapter Summary (2 of 3)

- A collection type can be specified by an interface
- A collection type can have several implementing classes
- Polymorphism is the use of the same operator symbol, function name, or method name in two or more implementations
- A bag collection type is unordered and supports operations to add, remove, and visit its items
- A class diagram is a visual notation for depicting relationships among classes
- Composition relates two classes as part to whole

Chapter Summary (3 of 3)

- Aggregation relates a class to another class as one to many
- The UML (Unified Modeling Language) is a graphical notation for depicting the relationships among software resources